Smart-X Ingestion Gateway

PROG7312 – Part 3 (Task 3) Written Answers

1. Implementation Report – Data Structure Explanations (Trees, Graphs, MST)
2. Project Completion Report
3. Technology Recommendations
4. Updates Made Based on Feedback from Parts 1 and 2

1. Data Structure Explanations: the Network Topology Feature

The Network Topology feature models the factory as a mesh: one **Gateway** (00:00:00:00:00:00), one **Router** per factory area (Assembly, Packaging, Boiler Room, Cold Storage, Paint Shop, Loading Bay, Press Shop), and every **Sensor** attached to its area's router. The operator sees this on the Topology page as a live map, can take any node offline, sees sensors "Rerouted via …" or "No route", opens a node's 2-hour history, and runs **Validate network**. Three families of structure carry this feature: trees, a weighted graph, and spanning-tree reasoning (MST / shortest-path tree).

1.1 Trees

a) AVL tree – TelemetryAvlTree (time-indexed history)

Role. When an operator selects a node on the Topology map, the details panel shows that sensor's readings for the last two hours. This is served by `GET /api/telemetry/{sensorId}/range?start=&end=`, which queries `TelemetryAvlTree`, a self-balancing binary search tree keyed on the reading timestamp.

Why it is efficient. An ordinary BST fed with telemetry degenerates into a linked list, because readings arrive in increasing time order (every insert goes right). The AVL tree rebalances after each insert using rotations, so its height stays at O(log n). Inserts therefore cost O(log n), and a range query costs O(log n + k) where k is the number of readings returned: the search prunes every subtree that lies wholly before *start* or after *end*, instead of scanning all 5,000 stored readings.

```
private static AvlNode Balance(AvlNode node)
{
    UpdateHeight(node);
    var balanceFactor = GetBalanceFactor(node);
    if (balanceFactor > 1)                      // left-heavy
    {
        if (GetBalanceFactor(node.Left!) < 0)    // left-right case
            node.Left = RotateLeft(node.Left!);
        return RotateRight(node);
    }
    if (balanceFactor < -1)                     // right-heavy (the common
    {                                           // case for time-ordered data)
        if (GetBalanceFactor(node.Right!) > 0)
            node.Right = RotateRight(node.Right!);
        return RotateLeft(node);
    }
    return node;
}
```

The range query only descends into a subtree when it can still contain matches:

```
private static void CollectRange(AvlNode? node, DateTime start, DateTime end, List<...> result)
{
    if (node is null) return;
    var timestamp = node.Data.Timestamp;
    if (timestamp >= start) CollectRange(node.Left, start, end, result);   // prune left
    if (timestamp >= start && timestamp <= end) result.Add(node.Data);
    if (timestamp <= end)   CollectRange(node.Right, start, end, result);  // prune right
}
```

Because results are collected in-order, they come back already sorted chronologically, so the client can draw the history sparkline without re-sorting. Memory is bounded: once the tree passes 5,000 readings, `RemoveMin` deletes the leftmost (oldest) node and rebalances on the way back up, still O(log n).

b) N-ary deployment tree – MeshDeploymentTree + DeploymentValidator

Role. "Validate network" checks that the live mesh is a correct **Gateway > Router > Sensor** hierarchy. `MeshDeploymentTree.Build` converts the topology into a tree of `DeploymentNode` objects, and

`DeploymentValidator.ValidateWithReasons` walks it recursively (depth-first, pre-order), naming every failing node.

```
void Visit(DeploymentNode? node, DeploymentNode? parent, int depth)
{
    if (node is null) { issues.Add(new(NodeId(parent), "Contains an empty child entry.")); return; }
    nodesChecked++;
    if (string.IsNullOrWhiteSpace(node.NodeName))      issues.Add(new(id, "Node name is missing."));
    else if (!seenNames.Add(node.NodeName.Trim()))     issues.Add(new(id, "Appears more than once in the tree."));
    if (string.IsNullOrWhiteSpace(node.ZoneId))        issues.Add(new(id, "Zone is missing."));
    else if (depth >= 2 && parent is not null && ... zones differ ...)
        issues.Add(new(id, $"Zone '{node.ZoneId}' does not match its parent's zone '{parent.ZoneId}'."));
    foreach (var child in node.Children ?? []) Visit(child, node, depth + 1);
}
```

Efficiency. Every node is visited exactly once, so validation is O(n). A `HashSet<string>` detects duplicate MAC addresses in O(1) per node rather than O(n) comparisons, keeping the whole check linear. Tree structure also lets the rule “a sensor must sit in the same zone as its router” be checked locally (child vs. parent), with no global search. The same parent/child hierarchy drives the “N issues downstream” roll-up on routers and the gateway in the client (`TopologyView`).

1.2 Graphs – MeshNetworkGraph (weighted, undirected, adjacency list) with Dijkstra

Role. The mesh itself is a graph: nodes are MAC addresses, edges are radio links with a cost (weight). Each sensor has a cheap link (cost 2) to its own router and a dearer backup link (cost 12) to the nearest neighbouring router; routers link to the gateway (cost = squared distance) and to their ring neighbours. Every route shown on the Topology page, every “Rerouted via” label, and every “No route” connection problem comes from this graph.

Representation. An adjacency list (`Dictionary<string, List<MeshEdge>>`) is used instead of an adjacency matrix because the mesh is sparse: each sensor has at most two links. Memory is O(V + E) rather than O(V²), and iterating a node’s neighbours costs only its degree. Offline nodes are held in a `HashSet`, so “is this node failed?” is O(1) and failing a node never mutates the edge list (restoring it is instant).

Shortest path. `FindShortestPath` is Dijkstra’s algorithm on .NET’s binary-heap `PriorityQueue`, giving O((V + E) log V). It skips offline nodes, so failing a router automatically produces the backup route. It stops as soon as the gateway is dequeued, and stale heap entries are discarded with a visited set (lazy deletion), which avoids the need for a decrease-key operation.

```
while (frontier.TryDequeue(out var current, out var distance))
{
    if (!visited.Add(current)) continue;            // stale entry
    if (IsSameMac(current, end.MacAddress)) break;  // early exit at the gateway
    foreach (var edge in _adjacencyList[current])
    {
        if (offline.Contains(edge.DestinationMac) || visited.Contains(edge.DestinationMac)) continue;
        var candidate = distance + edge.Weight;
        if (!distances.TryGetValue(edge.DestinationMac, out var known) || candidate < known)
        {
            distances[edge.DestinationMac] = candidate;
            previous[edge.DestinationMac] = current;
            frontier.Enqueue(edge.DestinationMac, candidate);
        }
    }
}
```

Detecting reroutes. The same method accepts `ignoreOffline: true`. `MeshNetworkService.GetRoute` runs Dijkstra twice, once on the live mesh and once as if every node were online (the baseline). If the two paths differ, the node is flagged `IsRerouted`; if the live path is empty the node is reported as “No route” and the background processor stops generating readings for it. This one graph therefore drives routing, failure simulation, the health bar’s connection problems and

the reporting/silent state of every sensor.

```
public MeshRoute GetRoute(string macAddress)
{
    var path     = _graph.FindShortestPath(macAddress, GatewayMac);
    var baseline = _graph.FindShortestPath(macAddress, GatewayMac, ignoreOffline: true);
    var cost     = path.Count > 1 ? _graph.GetPathCost(path) : 0;
    return new MeshRoute(path, baseline, cost);
}
```

1.3 Minimum Spanning Tree (MST)

Role in the design. A spanning tree of the mesh is the smallest set of links that still connects every node to the gateway. The mesh is deliberately *not* a tree: each sensor has a redundant backup link and routers form a ring, so there is always a cycle available for rerouting. The MST answers a different question from Dijkstra: “what is the cheapest set of links that keeps the whole factory connected?”, i.e. the primary backbone, while the remaining non-tree edges are the redundancy.

In this mesh the link costs are designed so that the backbone falls out naturally: a sensor’s own-router link (cost 2) is always cheaper than its backup link (cost 12), so Kruskal’s or Prim’s algorithm keeps the cost-2 link for every sensor and discards the backup – giving exactly the **Gateway > Router > Sensor** hierarchy that `MeshDeploymentTree` builds and the validator checks. That is why the Topology map draws sensors fanned around their own router: it is laying out the spanning backbone, and the dashed/secondary links are the cycle-forming edges.

Honest note on the implementation. The submitted code does not contain a separate Kruskal or Prim class. The spanning-tree behaviour is produced in two places instead: (1) `MeshDeploymentTree.Build` constructs the backbone tree directly from each sensor’s `ParentMac` (its cheapest link), and (2) Dijkstra’s `previous` map is a **shortest-path tree** rooted at the gateway. A shortest-path tree minimises each node’s distance to the gateway, whereas an MST minimises the total cost of all links; for this mesh’s cost scheme the sensor tier is identical in both. A Kruskal implementation that would fit the existing classes is shown below (it reuses `MeshGraphSnapshot`, which already returns each undirected edge once):

```
// Kruskal with union-find: O(E log E). Returns the backbone links.
public static List<MeshEdge> MinimumSpanningTree(MeshGraphSnapshot g)
{
    var parent = g.Nodes.ToDictionary(n => n.MacAddress, n => n.MacAddress,
                                      StringComparer.OrdinalIgnoreCase);
    string Find(string x) => parent[x] == x ? x : parent[x] = Find(parent[x]); // path compression
    var tree = new List<MeshEdge>();
    foreach (var e in g.Edges.Where(e => !g.OfflineNodes.Contains(e.SourceMac)
                                      && !g.OfflineNodes.Contains(e.DestinationMac))
                             .OrderBy(e => e.Weight))
    {
        var (a, b) = (Find(e.SourceMac), Find(e.DestinationMac));
        if (a == b) continue;          // would create a cycle -> redundant/backup link
        parent[a] = b;
        tree.Add(e);
    }
    return tree;                       // V-1 edges if the online mesh is connected
}
```

Contribution to efficiency. The MST gives the minimum-cost wiring for a deployment (useful when planning where to place routers), identifies which links are redundant, and if the online mesh yields fewer than V−1 tree edges it proves in a single O(E log E) pass that the network is partitioned – cheaper than running Dijkstra from every sensor, which is what `GetConnectionProblems` currently does (O(V · (V+E) log V)).

Structure	Topology feature it powers	Key complexity
AVL tree	Node details: 2-hour history / range query	Insert O(log n); range O(log n + k)
N-ary deployment tree + HashSet	Validate network; issues downstream	O(n) single DFS pass

Structure	Topology feature it powers	Key complexity
Weighted graph (adjacency list)	Map, links, failure/restore	O(V+E) memory; O(1) offline check
Dijkstra (binary heap)	Routes, cost, reroute and No-route detection	O((V+E) log V)
Spanning tree / MST	Gateway > Router > Sensor backbone and redundancy	Kruskal O(E log E)

2. Project Completion Report

2.1 Summary of the completed Smart-X project

Smart-X is an IoT telemetry ingestion gateway with a browser management console, built in .NET 8 as four projects in `SmartX.Gateway.slnx`:

- **SmartX.Gateway** – ASP.NET Core Minimal API server: REST endpoints, `TelemetryIngestionService`, `MeshNetworkService` and the `TelemetryBackgroundProcessor` hosted service; it also hosts the client.
- **SmartX.Gateway.Client** – Blazor WebAssembly console with Home, Sensors, Command Stream and Topology pages and a custom “Organic Precision” CSS design system.
- **SmartX.Gateway.Shared** – models, all data structures, thresholds (`SensorReadingInfo`), `CommandCatalog`, `SensorSearch`, deployment validation and the Predictive Action Engine, compiled once and used by both server and browser.
- **SmartX.Gateway.Tests** – xUnit, WebApplicationFactory and bUnit tests (152 tests at submission).

Part 1 delivered the foundation: the `SensorProfile` model with MAC-address regex validation, the generic `TelemetryReading<T>` wrapper constrained to value types, the recursive deployment-tree validator, thread-safe ingestion with locks, minimal API endpoints and the first dashboard. **Part 2** added the ingestion pipeline data structures: FIFO `TelemetryMessageQueue`, min-heap `CriticalAlertPriorityQueue` so warning/critical readings bypass normal traffic, `CommandHistoryStack` for undo, `DeviceRegistry` dictionary for O(1) lookup, `SortedTelemetryLog`, `ActiveAlertSet` for alert de-duplication, the background processor and the Predictive Action Engine. **Part 3** added the Network Topology feature (AVL tree, mesh graph with Dijkstra, deployment-tree validation of the live mesh), realistic telemetry simulation, usability work across all pages, and the automated test project.

2.2 Architectural and algorithmic challenges in Task 3, and how they were overcome

Challenge 1 – Integrating the C# backend with the Blazor WebAssembly frontend

Server and client run in different places: the API on the server, the Blazor client inside the browser. Early on, the console and the server each held their own copies of rules such as “what counts as critical”, units per sensor type, and which commands are valid for which device. These drifted apart (for example, the UI could offer a valve command to a power meter that the server then rejected). **Solution:** all shared rules were moved into the `SmartX.Gateway.Shared` class library (`SensorReadingInfo`, `CommandCatalog`, `FactoryAreas`, `SensorSearch`), referenced by both projects. Because Blazor WebAssembly runs real .NET in the browser, the exact same C# executes on both sides and the DTOs (`MeshTopology`, `MeshTopologyNode`) serialise and deserialise without hand-written mapping. The server registers the client assembly with `AddAdditionalAssemblies(typeof(SmartX.Gateway.Client._Imports).Assembly)` so one process serves both.

Challenge 2 – Stale data in the browser

Live pages sometimes showed old readings because responses were being cached. **Solution:** a small middleware sets `Cache-Control: no-store` on every `/api` response, and the Topology page polls every 5 seconds with a `System.Threading.Timer` that is disposed when the page is left, with a “Pause updates” option so a selected node does not jump while the operator reads it.

Challenge 3 – Time zones

The gateway stamps everything in UTC but operators expect local time, and JSON round-trips can lose the `DateTimeKind`. **Solution:** a client-side `TimeDisplay` helper treats unspecified timestamps as UTC before converting, and the "relative time" label tolerates small clock differences between gateway and browser (anything under 5 s is "just now").

Challenge 4 – Detecting reroutes, not just routes

Dijkstra returns the current best path, but the UI also needed to say *whether* a sensor was rerouted. **Solution:** `FindShortestPath` gained an `ignoreOffline` flag; comparing the live path with the all-online baseline flags reroutes without storing any extra state, and failure is modelled as a `HashSet` of offline nodes rather than deleting edges, so restore is O(1).

Challenge 5 – Concurrency

The API thread pool and the background processor touch the same structures at the same time. **Solution:** every data structure owns a private lock and returns copies/snapshots (`GetSnapshot()`) rather than live collections, so the UI never enumerates a collection that is being modified.

Challenge 6 – Unbounded memory and degenerate trees

Readings arrive in time order, which would turn a plain BST into a linked list, and continuous simulation would grow memory without limit. **Solution:** an AVL tree (self-balancing) and a cap of 5,000 readings in both the AVL index and the sorted log, with 200 readings per sensor.

Challenge 7 – Laying out a readable map

With up to 60 sensors the map became cluttered. **Solution:** routers sit on a fixed ring (one angle per area) and sensors fan outwards in concentric arcs, each ring holding two more sensors than the one inside; narrow screens switch automatically to a list view.

Challenge 8 – Building the .slnx solution

The new `.slnx` solution format cannot be built from the command line on older .NET 8 SDKs. **Solution:** build and run commands target the server `.csproj` directly (which restores Client and Shared through project references), guaranteeing the project compiles and runs for the marker.

2.3 Key learnings

- **Choosing structures for the access pattern, not by habit.** The priority queue was chosen because critical readings must jump the queue; the HashSet because a sensor must raise at most one open alert; the AVL tree because the history panel asks for time *ranges*. I learned to start from "what question does this screen ask?" and choose the structure that answers it fastest.
- **Graph algorithms in a real setting.** Implementing Dijkstra over MAC-addressed nodes taught practical details textbooks skip: lazy deletion instead of decrease-key, early exit at the target, and modelling failure as a filter rather than mutating the graph.
- **Self-balancing trees.** Writing the four AVL rotation cases by hand, and then discovering that rotations can move a duplicate timestamp to either side, which is why the range query explores both subtrees on equality.
- **Recursion.** The deployment validator moved from "return false at the first error" (Part 1) to collecting every failing node with a reason, which is far more useful to an operator.
- **Full-stack .NET.** Blazor WebAssembly, Minimal APIs with typed results (`Results<Ok<T>, BadRequest<string>>`), dependency injection of singletons, and `BackgroundService` hosted

workers.

- **Thread safety.** Locks, snapshots and avoiding exposure of live collections across threads.
- **Testing.** Using `WebApplicationFactory` to host the API in memory with the background generator removed, and seeded `Random` instances so simulation tests are repeatable.
- **Problem-solving approach.** Reproduce, isolate, fix at the root (e.g. moving shared rules into one library instead of patching the UI), then add a test so the bug cannot return.
- **Usability.** Designing for operators: plain-language status, colour *and* icon for each state (accessibility), confirmation before destructive actions, and showing what Undo will undo.

3. Technology Recommendations

Technology	What it adds to Smart-X	Justification / .NET compatibility
CI/CD with GitHub Actions	Run `dotnet build` and the 152 `dotnet test` tests on every push/PR using the official `actions/setup-dotnet` action; publish to Azure or a container registry on merge to main.	The repository is already on GitHub and the solution is pure .NET 8 with an xUnit project, so the pipeline is a few lines of YAML. It guarantees the code always compiles and runs, which is a hard requirement of this project.
SignalR for real-time push	Replace the 5-second polling on the Topology and Command Stream pages with server push of new readings, alerts and node failures.	SignalR is part of ASP.NET Core (already referenced with `AddSignalR()` in Program.cs) and has a Blazor WebAssembly client. It cuts network traffic and makes alerts appear instantly.
Persistent database: Azure SQL / PostgreSQL with EF Core, or Google Cloud Spanner/Firestore	Persist sensors, telemetry, command history and topology so data survives restarts (currently all in memory).	Entity Framework Core has first-class providers for SQL Server, PostgreSQL (Npgsql) and Google Cloud Spanner. The in-memory structures can stay as a fast cache in front of the database.
Time-series database (InfluxDB or Azure Data Explorer)	Store high-volume telemetry for long-term history and range queries.	Both have official .NET client libraries; they complement the in-memory AVL index, which is capped at 5,000 readings.
MQTT broker (MQTTnet / Azure IoT Hub)	Let real devices publish telemetry over MQTT, the standard IoT protocol, rather than HTTP POST.	MQTTnet is a mature .NET library; Azure IoT Hub has a .NET SDK and adds device identity and provisioning.
LLM API for daily alert summaries	A scheduled job sends the day's alerts, commands and insights to an LLM API (e.g. Anthropic Claude or Azure OpenAI) and shows a plain-English summary on the Home page.	Official .NET SDKs exist; the job fits the existing `BackgroundService` pattern and extends the Predictive Action Engine.
Home Assistant integration	Expose Smart-X sensors and commands to Home Assistant via its REST/MQTT integration for dashboards and automations.	Home Assistant speaks MQTT and REST, which ASP.NET Core and MQTTnet already support.
Docker + Kubernetes / Azure Container Apps	Containerise the gateway for repeatable deployment and horizontal scaling.	.NET 8 ships official container images and supports `dotnet publish /t:PublishContainer` with no Dockerfile.
Redis (distributed cache/queue)	Share alert sets, queues and caches between multiple gateway instances.	StackExchange.Redis is the standard .NET client; Redis has native sorted sets and lists matching our queue and log structures.
OpenTelemetry + Application Insights / Grafana	Trace and measure ingestion latency, queue depth and error rates.	OpenTelemetry .NET is built into ASP.NET Core instrumentation.
ASP.NET Core Identity / Microsoft Entra ID	Authenticate operators and authorise commands (e.g. only supervisors may take routers offline).	Built into ASP.NET Core with Blazor WebAssembly authentication support.

Recommended order: (1) GitHub Actions CI/CD, as it protects the compile-and-run requirement immediately; (2) SignalR, as it is already referenced and removes polling; (3) a persistent database via EF Core; (4) MQTT ingestion for real devices; (5) the LLM daily summary and Home Assistant integration as value-adding features.

4. Updates Made Based on Feedback from Parts 1 and 2

Area	Update made
Shared business rules	Thresholds, units and status rules moved into `SensorReadingInfo` in the Shared project so the server and console always agree (Critical <5 or >95, Warning <10 or >90).
Command validation	`CommandCatalog` only allows commands that fit the device type (valves on actuators, relays on power meters, restart on anything); the API returns 400 otherwise and the UI only offers valid commands.
Commands have visible effects	Restart Sensor returns the reading to normal; Open/Close Valve moves an actuator by 15 points; Undo reverses it and states what it will undo.
Deployment validation	New `ValidateWithReasons` reports every failing node with a reason (missing name/zone, duplicates, zone mismatch) instead of only true/false; handles null children.
Alerts	Alert details (value, priority, time), Acknowledge and Acknowledge all; an alert stays cleared until the next bad reading.
Realistic telemetry	Readings every 4–12 s using a random walk pulled back to normal with occasional fault spikes; pause/resume of live data.
Memory limits	Telemetry log and AVL index capped at 5,000 readings; 200 readings per sensor; `?readings=N` on the sensors API.
Usability	Reworked Home page with plain-language summary and getting-started guide; health summary bar; sparklines; status pills with colour and icon; sortable/searchable Sensors table; nav alert badge; local-time display.
Search	Device search ignores case and separators and accepts the last six hex digits of a MAC address.
Sensor sources	Mesh, Demo and Manual sensors tagged; “Clear all sensors” keeps and resets mesh sensors; seed count validated (1–50).
Caching	`Cache-Control: no-store` on all API responses so the console never shows stale data.
Testing	New xUnit test project (152 tests) covering data structures, validation, catalog/search, ingestion service and API endpoints.
Documentation	README updated with build/run steps that work on the .NET 8 SDK, full API reference and usage walkthrough.
New in Part 3	Network Topology page, mesh graph with Dijkstra rerouting, node failure/restore, AVL range queries, Validate network.